

Document: Technical Project Submission Report

Project Name: Supervised Machine Learning & Data Classification

Intern Name: Dridi Jawher

Date of Submission: May 23, 2026

Project 2 Architecture & Machine Learning Pipeline

This report defines the mathematical and predictive pipeline engineered for **Project 2: Supervised Learning & Data Classification**. Transitioning from heuristic rule-based frameworks, this system introduces automated feature pattern derivation using the classic **Iris Plant Dataset Benchmark**. The core classification infrastructure leverages a K-Nearest Neighbors (KNN) algorithm with feature equalization matrices.

1. The Analytical Data Pipeline

To ensure complete structural integrity and eliminate spatial scale bias during neighbor metric computing, the layout maps data strictly through an Input-Process-Output framework:

Pipeline Step	Operation & Parameter Definitions	Strategic Rationale
1. Dataset Acquisition	150 total balanced samples; 4 physical morphological dimensions.	Provides accurate learning grounds across 3 plant species targets.
2. Data Shuffling & Split	80/20 train-test ratio randomization matrix.	Erases dataset ordering bias to ensure general validation metrics.
3. Feature Scaling	Standardization via <code>StandardScaler()</code> (Mean=0, Variance=1).	Prevents dominant distance calculation distortion from high magnitudes.
4. Classifier Fit	K-Nearest Neighbors Algorithm tuned explicitly to <code>n_neighbors=5</code> .	Optimal classification boundaries while preventing over-fitting.

2. Implementation Source Code

The code block below outlines the supervised machine learning sequence, built with integrated mathematical verification callbacks:

```
# =====
# DECODELABS ARTIFICIAL INTELLIGENCE TRACK: PROJECT 2
# MACHINE LEARNING ENGINE: K-NEAREST NEIGHBORS (IRIS BENCHMARK)
# =====

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, f1_score

print("🤖 MODEL LIFE-CYCLE STARTED: TUNING HYPERPARAMETERS...")

# 1. ACQUIRE TARGETS AND SAMPLES
iris_dataset = load_iris()
Features, Targets = iris_dataset.data, iris_dataset.target

# 2. RANDOMIZED SHUFFLE & METRIC INTEGRITY SPLIT (80% Train / 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    Features, Targets, test_size=0.20, random_state=42, shuffle=True
)

# 3. COMPLY WITH GATEKEEPER RULES: Z-SCORE STANDARDIZATION
feature_scaler = StandardScaler()
X_train_scaled = feature_scaler.fit_transform(X_train)
X_test_scaled = feature_scaler.transform(X_test)

# 4. INSTANTIATE LOGIC MODEL ENGINE (K=5 Neighbors)
classification_model = KNeighborsClassifier(n_neighbors=5)
classification_model.fit(X_train_scaled, y_train)

# 5. EXECUTE PREDICTION BATCH OVER TEST CHANNELS
predicted_vectors = classification_model.predict(X_test_scaled)

# 6. OUTPUT METRIC EVALUATION REPORTING
print("📋 Resulting Confusion Matrix Evaluation:")
print(confusion_matrix(y_test, predicted_vectors))

macro_f1_result = f1_score(y_test, predicted_vectors, average='macro')
print(f"🎯 Achieved Macro F1 Score Boundary: {macro_f1_result:.4f}")
```

Quality Validation Summary: The matrix reports zero cross-class contamination boundaries, maintaining an optimal Macro F1-score close to 1.00. This confirms high performance across all evaluated dimensional boundaries.

3. Conclusion & Theoretical Affirmation

The implemented System 1 Predictive Engine successfully replaces explicit programming architectures with statistical supervised learning frameworks. This model establishes a fully auditable classification structure, ready for integration into higher-dimensional neural topologies.